

Strings:

String is a collection of characters enclosed between pair of single quotes or pair of double quotes or pair triple quotes.

Syntax:

```
Var = 'val1val2val3.....'  
or  
Var = "val1val2val3....."  
or  
Var = """va1val2val3...."""
```

E.X: greet = 'welcome to python class'

greet = "welcome to python class"

sentence = """hello everyone

how are you"""

- Where values can be uppercase alphabets or lowercase alphabets or numbers or any special characters.
- When we create the string with single quotes then we have to end it with the single quotes and same for double and triple quotes.
- If we create the string with single quote or double quote or triple quote it will internally converted to single quote.
- Triple quote string is used to write the multiple lines string.
- If single quote is already present in the original string then we can represent the string in two forms
 - Using backslash(\): By using the backslash we can remove the special behaviour of the quotes.(not a pythonic way)
 - Using double or triple quote.
- If double quote is already present in the original string then we can represent the string in two forms
 - Using backslash(\)
 - Using single or triple quote

Formatting string:

It is a process of inserting a custom string/variable in predefined text. We can write format string in two forms:

- Using curly brackets {}.
- Using format attribute.

Length function:

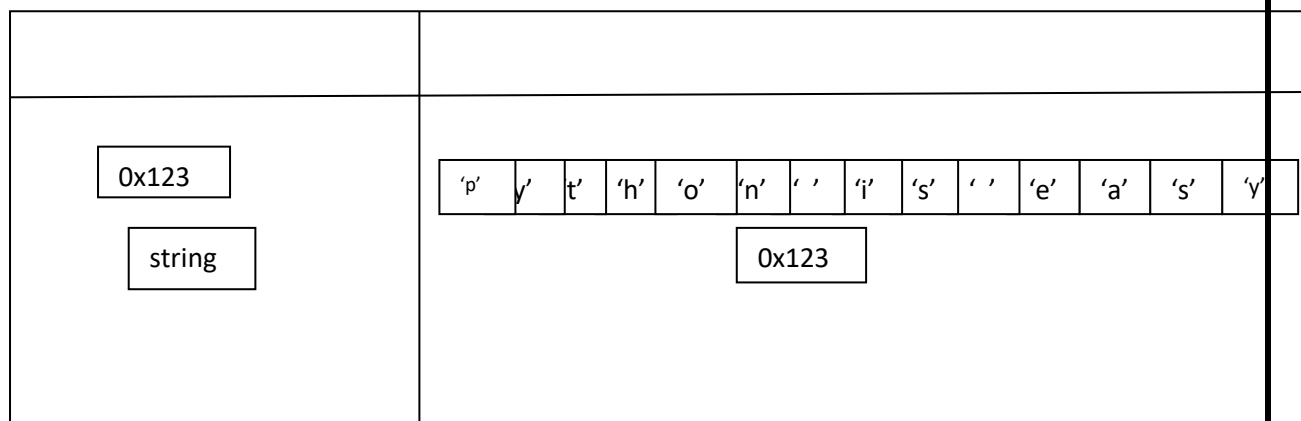
It is a function, which is used to find the number of values present inside the collection.

Syntax:

len(variable/value)

Memory allocation for collection data type:

- As soon as , the control gets to know variable creation process it divide the memory into two space
 - Variable space
 - Value space
- When control gets to know that the value is collection it create the layer of memory in the value space.
- Layer will be divided into number of the blocks, which is exactly equal to the length of the collection.
- Address will be given to the created layer and that address will be stored with respect to the variable name in variable space.
- **Ex:** string = 'python is easy'



- **dir ():** it is used to get the list of all the attributes present in the given class.
- **Attributes:** something that can be accessed by the dot('.') operator.

Methods of string:

1. **Capitalize ()**: python strings capitalize () function returns the first letter of the string in the capital form.
 - Syntax: Var.capitalize()
2. **Startswith ()**: python strings Startswith () function returns True if the string starts with the given prefix, otherwise it returns False.
 - Syntax: Var.startswith('character/substring')
3. **Endswith ()**: python strings endswith () function returns True if the string ends with the given suffix, otherwise it returns False.
 - Syntax: Var.endswith('character/substring')
4. **Isalpha ()**: python string isalpha () function returns True if all the characters in the string are alphabets, otherwise False.
 - Syntax: Var.isalpha()
5. **Isalnum ()**: python string isalnum () function returns True if all the characters in the string are alphanumeric only, otherwise False.
 - Syntax: Var.isalnum()
6. **Isdecimal ()**: python string Isdecimal () function returns True if all the characters in the string are decimal ('0' to '9') characters, otherwise False.
 - Syntax: Var.isdecimal()
7. **Isdigit ()**: python string isdigit () function returns True if all the characters in the string are digit ('0' to '9') characters, otherwise False.
 - Syntax: Var.isdigit()
8. **Islower ()**: python string Islower () returns True if all the character in the string are lowercase and if any one character is uppercase it will return False.
 - Syntax: Var.islower()
9. **Isupper ()**: python string Isupper () returns True if all the character in the string are uppercase and if any one character is lowercase it will return False.
 - Syntax: Var.isupper()
10. **Isnumeric ()**: python string Isnumeric () returns True if all the character in the string are numeric, otherwise False.
 - Syntax: Var.isnumeric()
11. **Isspace ()**: python string isspace () returns True if there are only whitespace characters in the string, otherwise it returns False.
 - Syntax: Var.isspace()
12. **Istitle ()**: python string istitle () returns True if each word in the string is uppercase, otherwise it returns False.
 - Syntax: Var.istitle()
13. **Upper ()**: python string upper () returns all the alphabet characters to uppercase.(we can have numbers and special character in string).

- Syntax: Var.upper()
14. **Lower ()**: python string lower () returns all the alphabet characters to lowercase.(we can have numbers and special character in string).
 - Syntax: Var.lower()
 15. **Swapcase ()**: python string Swapcase () returns new string which convert uppercase to lowercase and vice-versa.
 - Syntax: Var.swapcase()
 16. **Title ()**: python string title () returns new string convert first character of each word to uppercase.
 - Syntax: Var.title ()
 17. **Strip ()**: python string strip () used to trim whitespace or the character which we pass as a delimiter from the string.
 - Syntax: Var.strip ()
 18. **Rstrip ()**: python string Rstrip () used to trim whitespace or the character which we pass as a delimiter only at right hand side from the string.
 - Syntax: Var.rstrip ()
 19. **Lstrip ()**: python string Lstrip () used to trim whitespace or the character which we pass as a delimiter only at left hand side from the string.
 - Syntax: Var.lstrip ()
 20. **Format ()**: python string format () used to create the formatted string from the template string and the supplied value.
 21. **Replace ()**: python string replace () is used to create a new string by replacing some parts of another string.
 - Syntax: Var.replace(old_string, new_string)
 22. **Split ()**: python string split () is used to split a string into list of string based on the delimiter.
 - Syntax: Var.split ('delimiter') ## by default it takes as whitespace
 23. **Find ()**: python string find () is used to find the lowest index of a substring in a string. If the substring is not present in the given string it will give -1 as output
 - Syntax: Var.find ('character')
 24. **Index ()**: python string index () returns the lowest index of the specified substring. If the substring not present in the given string it will throw the error message
 - Syntax: Var.index('character')
 25. **Count ()**: python string count () returns the number of occurrence of the specific substring inside the given string.
 26. **Rfind ()**: python string find () is used to find the highest index of a substring in a string. If the substring is not present in the given string it will give -1 as output
 - Syntax: Var.rfind (character)
 27. **Rindex ()**: python string index () returns the highest index of the specified substring. If the substring not present in the given string it will throw the error message
 - Syntax: Var.rindex('character')

Indexing of the string:

Indexing is a sub-address given to each and every block of the memory present inside the given collection. To access the individual value from the collection we need to use indexing.

There are two types of indexing:

1. **Positive indexing:** Positive indexing is used when we want to transverse from left to right of the collection. And it starts from 0.
2. **Negative indexing:** Negative indexing is used when we want to transverse from right to left of the collection. And it starts from -1.

The syntax used to access individual value using indexing is

Variable[index]

str="PyTHON"

P	y	T	H	O	N
---	---	---	---	---	---

Positive index $\Rightarrow$ 0 1 2 3 4 5

Negative index $\Rightarrow$ -6 -5 -4 -3 -2 -1

- For the user convenience there are types of indexing but control will understand only positive indexing.
- The formula used to convert negative index to positive index is

$$\text{Positive_index} = \text{length}(\text{collection}) + (\text{negative_index})$$

The syntax used to modify any collection is:

Variable[index] = new_value

Based on modification of values, collection got classified into two types

- Mutable data-type:** It is defined as, if any data item allows the user to modify the original values or the characters or elements in it.
- Immutable data-type:** It is defined as, if any data item does not allows the user to modify the original values or the characters or elements in it.

Since, strings not allow users to modify the original characters in it we consider string as a immutable collection.

